

4.5 GBM

1. GBM 개요 및 실습

- 부스팅 알고리즘
 - 여러 개의 약한 학습기를 순차적으로 학습, 예측하면서 잘못 예측한 데이터에 가중치 부여를 통해 오류를 개선해 나가면서 학습하는 방식
 - 대표적 구현: 에이다 부스트 / 그래디언트 부스트
- 경사 하강법
 - 오류 값: 실제 값 - 예측값
 - 오류 식: $h(x) = y = F(x)$
 - 오류식을 최소화하는 방향성을 가지고 반복적으로 가중치 값을 업데이트하는 것
- GBM이 랜덤 포레스트보다는 예측 성능이 뛰어난 경우가 많음
- but 수행 시간이 오래 걸리고, 하이퍼 파라미터 튜닝 노력도 더 필요함

2. GBM 하이퍼 파라미터

- loss: 경사 하강법에서 사용할 비용 함수를 지정, 특별한 이유가 없으면 기본값인 'deviance'를 그대로 적용
 - learning_rate: GBM이 학습을 진행할 때마다 적용하는 학습률, Weak learner가 순차적으로 오류 값을 보정해 나가는 데 적용하는 계수
 - n_estimators: Weak learner의 개수, 많을수록 수행 시간이 오래 걸리며, 기본값은 100
 - subsample: Weak learner 학습에 사용하는 데이터의 샘플링 비율, 기본값은 1
-

4.6 XGBoost (eXtra Gradient Boost)

1. XGBoost 개요

- GBM을 기반으로 성능과 속도를 개선한 알고리즘
- 병렬 CPU 환경에서 병렬 학습이 가능해 기존 GBM보다 빠르게 학습을 완료
- 핵심 라이브러리는 C/C++로 작성됨
- 파이썬 패키지명은 xgboost

장점

- 뛰어난 예측 성능
- GBM보다 빠른 수행 시간
- 과적합 규제

- 나무 가지치기
- 자체 내장된 교차 검증
- 결손값 자체 처리

2. XGBoost 파라미터 종류

① 일반 파라미터

- booster
- silent
- nthread

② 부스터 파라미터

- eta
- num_bosst_rounds
- min_child_weight
- gamma
- max_depth
- sub_sample
- colsample_bytree
- lambda
- alpha
- scale_pos_weight

③ 학습 태스크 파라미터

- objective
- binary:logistic
- multi:softmax
- multi:softprob
- eval_metric
 - rmse, mae, logloss, error, auc, merror, mlogloss

과적합이 심한 경우:

- eta 값↓
- max_depth 값↓
- min_child_weight 값↑
- gamma 값↑
- subsample, colsample_bytree 조정

조기종단기능 **Early Stopping**

- 검증 데이터의 성능이 더 이상 좋아지지 않으면 학습을 조기에 종료
- 불필요한 반복 학습 방지

파이썬 래퍼 **XGBoost**

- XGBoost만의 전용 데이터 객체인 **DMatrix**를 사용
 - 주요 입력 파라미터는 **data**와 **label**
 - **data**는 피쳐 데이터 세트이며, **label**은 분류의 경우에는 레이블 데이터 세트, 회귀의 경우는 숫자형인 종속값 데이터 세트임
 - 하이퍼 파라미터를 **xgboost** 모듈의 **train()** 함수에 파라미터로 전달함
 - 수행 속도를 개선하기 위해서 조기 중단 기능을 제공
 - 성능평가는 주로 별도의 검증 데이터 세트를 이용
 - 데이터 세트에 대한 교차 검증 수행 후 최적 파라미터를 구할 수 있는 방법을 **cv()** API로 제공
 - 사이킷런과 호환돼 편리하게 사용 가능
-

4.7 LightGBM

1. LightGBM 개요

- XGBoost와 마찬가지로 대용량 데이터에 대한 높은 예측 성능을 가지며, 병렬 컴퓨팅 기능 제공
- XGBoost보다 학습에 걸리는 시간이 훨씬 적음
- 리프 중심 트리 분할(Leaf Wise) 방식 사용
- 파이썬 패키지명 'lightgbm'

2. LightGBM 주요 파라미터

- num_iterations
- learning_rate
- max_depth
- min_data_in_leaf
- num_leaves
- boosting
 - gbdt
 - rf
- bagging_fraction
- feature_fraction
- lambda_l2
- lambda_l1

하이퍼 파라미터 튜닝 방안

- `num_leaves` 개수 ↑ → 모델 복잡도 ↑ → 과적합 가능성 ↑
- `min_child_samples` 값 ↑ → 트리의 지나친 분할 억제
- `max_depth` ↓ → 트리 깊이 제한
- `learning_rate` ↓ + `n_estimators` ↑
- `reg_lambda`, `reg_alpha` → 규제
- `subsample`, `colsample_bytree` → 과적합 조절

3. 파이썬 래퍼 **LightGBM**과 사이킷런 래퍼 **XGBoost**, **LightGBM** 하이퍼 파라미터 비교

- 분류 → `LGBMClassifier`
- 회귀 → `LGBMRegressor`

XGBoost와 마찬가지로 **Early Stopping** 사용 가능

4.8 베이지안 최적화 기반 **HyperOpt**를 이용한 하이퍼 파라미터 튜닝

1. 베이지안 최적화 개요

- 최대 또는 최소 함수 반환 값을 만드는 최적 입력값을 가능한 적은 시도를 통해 빠르고 효과적으로 찾아주는 방식
- 베이지안 확률에 기반을 두고 있는 최적화 기법
- 대체 모델(Surrogate Model)
 - 획득 함수로부터 최적 함수를 예측할 수 있는 입력값을 추천 받은 뒤 이를 기반으로 최적 함수 모델을 개선
- 획득 함수(Acquisition Function)
 - 개선된 대체 모델을 기반으로 최적 입력값을 계산

HyperOpt 사용하기

1. 검색 공간 **Search Space** 설정
2. 목적 함수 **Objective Function** 설정
3. 목적 함수의 반환 최솟값을 가지는 최적 입력값 유추

3. XGBoost 하이퍼 파라미터 최적화

- 하이퍼 파라미터 입력 시 형변환이 필요함
- 성능 값이 클수록 좋은 성능 지표일 경우 -1을 곱해 줘야 함
 - 더 큰 성능 지표가 더 작은 반환값이 되도록

4.10 분류 실습 — 캐글 신용카드 사기 검출

1. 언더 샘플링과 오버 샘플링의 이해

- 레이블이 불균형한 분포를 가진 데이터 세트를 학습시킬 때 예측 성능의 문제가 발생
 - 이상 레이블을 가지는 데이터 건수가 정상 레이블을 가진 데이터 건수에 비해 너무 적기 때문 -> 적절한 학습 데이터를 확보하는 방안이 필요
 - 오버 샘플링 (Oversampling): 적은 데이터 세트를 증식하여 학습을 위한 충분한 데이터를 확보하는 방법
 - 언더 샘플링 (Undersampling): 많은 데이터 세트를 적은 데이터 세트 수준으로 감소시키는 방식
-

4.11 스택킹 앙상블

- 개별적인 여러 알고리즘을 서로 결합해 예측 결과를 도출
 - 개별 알고리즘으로 예측한 데이터를 기반으로 다시 예측을 수행
- 개별적인 기반 모델 & 해당 기반 모델의 예측 데이터를 학습 데이터로 만들어서 학습하는 최종 메타 모델이 필요
- 일반적으로 성능이 비슷한 모델을 결합해 좀 더 나은 성능 향상을 도출하기 위해 적용됨

1. 기본 스택킹 모델

예제에서 사용한 개별 모델:

- KNN
- Random Forest
- Decision Tree
- AdaBoost

최종 메타 모델:

- Logistic Regression

과정

1. 여러 개별 모델 학습
2. 각각 예측 수행

3. 각 모델의 예측값을 하나로 결합
4. 결합된 데이터를 메타 모델의 학습 데이터로 사용
5. 메타 모델이 최종 예측

2. CV 세트 기반 스택킹

기본 스택킹의 문제:

- 테스트용 레이블 데이터 세트를 기반으로 학습했기에 과적합 문제가 발생할 수 있음

→ 이를 개선하기 위해 교차 검증(CV) 기반 스택킹 사용

- 스택킹 모델은 분류(Classification)뿐만 아니라 회귀(Regression)에도 적용 가능